



Computer Networks

Week 2 — Detailed Study Notes

Circuit & Packet Switching · Layering & Protocols · Measuring Networks:
the Four Delays, Bandwidth-Delay Product, Queues & Little's Law

IIT Madras BS — Prof. Ayon Chakraborty

How to read these notes. Every slide concept from the five Week-2 lectures appears here, in slide order, grouped by module (Module 2. n = Lecture n). Box types: **Definition**, **Intuition**, **Formula** (derivations + [units]), **Common pitfall**, **Beyond the slides** (standard Kurose & Ross material, flagged as extra), and grey **What the slide says**. The end has a formula sheet, numerical problems with full solutions, and MCQs with explanations.

Contents

1	Module 2.1 — Circuit Switching	3
1.1	Where we left off	3
1.2	The four steps of circuit switching	3
1.2.1	A circuit inside a switch	4
1.3	Many kinds of “circuits”: TDM and FDM	4
1.4	Timing in circuit switching	5
1.4.1	Three timing scenarios on the slides	6
1.5	Circuit switching doesn’t “route around trouble”	7
1.6	Circuit switching: pros and cons	7
2	Module 2.2 — Packet Switching	8
2.1	How packet switching works	8
2.2	Packet switching: pros and cons	9
2.3	Two approaches compared — and a hybrid	9
2.4	The Internet uses store-and-forward packet switching	9
3	Module 2.3 — The Concept of Layering	10
3.1	Different functionalities in different places	11
3.2	The layered stack	11
3.3	Protocols: the rules of networking	12
3.4	The protocols of the Internet	13
3.5	Encapsulation: the packet takes on headers at each layer	13
3.6	Routers have network and link layers too!	14
4	Module 2.4 — Why Layering, and Measuring Networks	15
4.1	Why layering?	15
4.2	This course has layers	16
4.3	Measuring networks: definitions	17
5	Module 2.5 — The Four Components of Delay	17
5.1	1. Propagation delay	17
5.2	Bandwidth-delay product	17
5.3	2. Transmission delay	18
5.4	Packet delay: the slide’s worked example	19
5.5	Packet delay: the “pipe” view	19
5.6	3. Queuing delay	20
5.6.1	The pipe view of queuing (slides 13–20)	21
5.6.2	Queue length over time	22

5.6.3	What queuing delay depends on	22
5.7	Basic queuing theory terminology	23
5.8	Little's Law (John Little, 1961)	24
5.8.1	Why Little's Law is true — a derivation	24
5.8.2	Slide example	25
5.8.3	Applying Little's law to networks — which “system”?	25
5.9	4. Processing delay	25
5.10	Recap on delay	25
6	Formula Sheet	26
7	Numerical Problems with Worked Solutions	26
8	Multiple-Choice Questions with Explanations	29

1 Module 2.1 — Circuit Switching

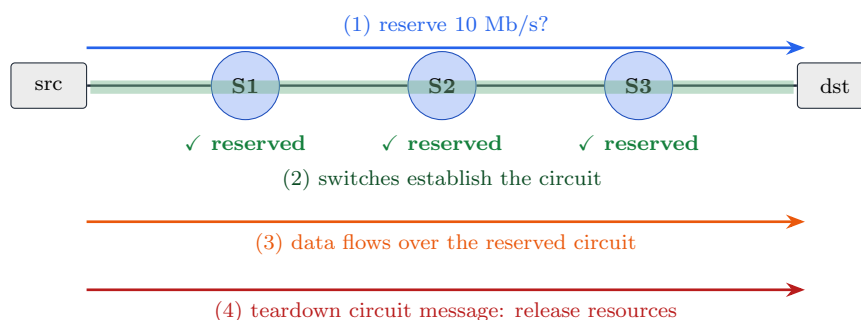
1.1 Where we left off

What the slide says (W2 L1, slide 1)

Two approaches to sharing: **Reservations** → **circuit switching**; **On demand** → **packet switching**. *How are these implemented?*

Week 1 established *why* on-demand sharing is more efficient for bursty traffic (statistical multiplexing). Week 2 asks *how* each approach is actually built, what it costs in time, and what it does when things fail.

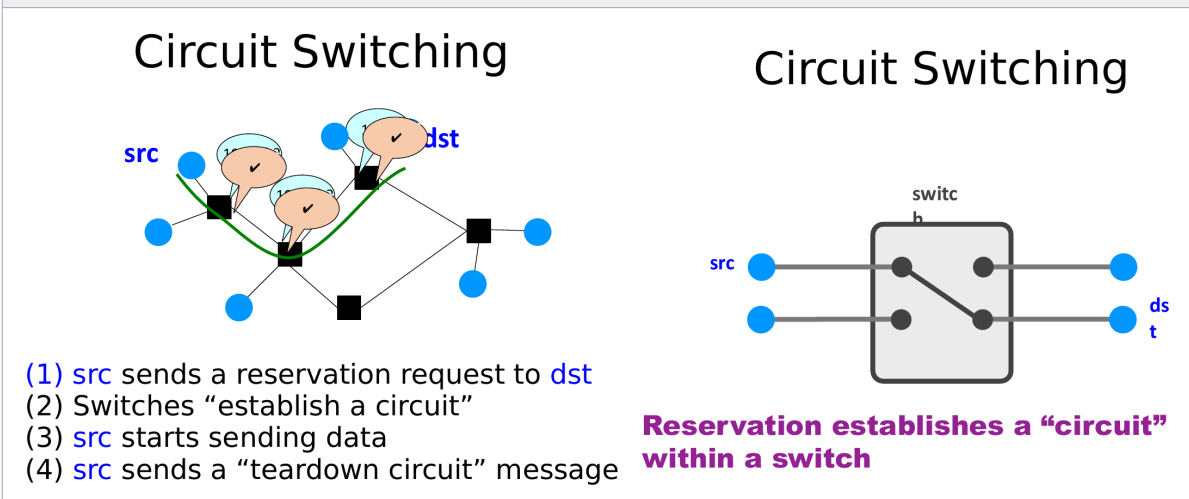
1.2 The four steps of circuit switching



What the slide says (W2 L1, slide 2)

(1) src sends a reservation request to dst; (2) switches “establish a circuit”; (3) src starts sending data; (4) src sends a “teardown circuit” message.

From the lecture slides: W2 L1, slides 2–3 — reserve the path; the reservation creates a “circuit inside each switch”



Definition: Circuit

An end-to-end path along which *every* switch has reserved a fixed share of capacity (a time slot or a frequency band on each outgoing link) for one conversation, for the whole duration of that conversation — whether or not data is flowing.

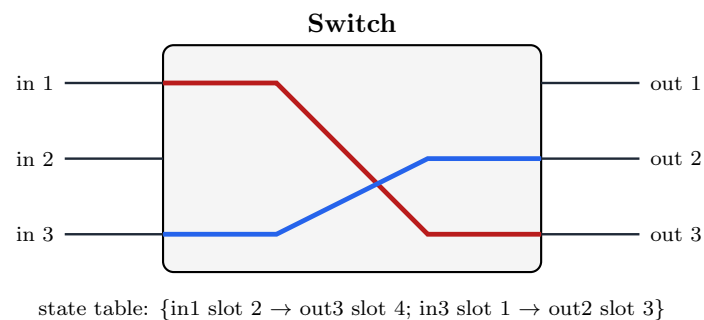
Intuition: Booking a train compartment

Step 1 is making the booking request at every station on the route; step 2 is each station confirming it has held seats; step 3 is travelling — no queue at any station, your seats are waiting; step 4 is cancelling so the seats return to the pool. If any one station says “full”, the whole booking fails (*call blocked*, “all lines busy”).

1.2.1 A circuit inside a switch

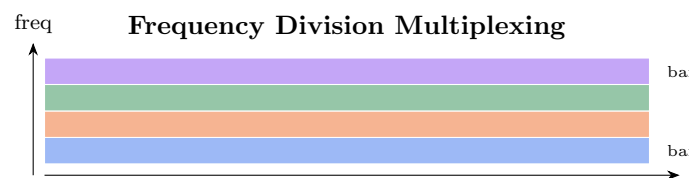
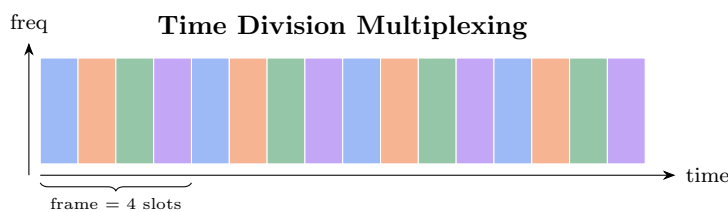
What the slide says (W2 L1, slide 3)

Reservation establishes a “circuit” *within* a switch.



The switch keeps **per-circuit state**: a table mapping (input port, slot) → (output port, slot). Once filled in, forwarding is trivial — no header lookup, just “whatever arrives in slot 2 on port 1 goes out in slot 4 on port 3”. This is why circuit switching is “simple/fast once the circuit is established”.

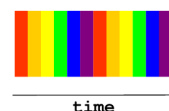
1.3 Many kinds of “circuits”: TDM and FDM



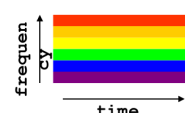
From the lecture slides: W2 L1, slide 4 — TDM (time slots) and FDM (frequency bands)

Many kinds of “circuits”

- Time division multiplexing
 - divide time in *time slots*
 - separate time slot per circuit



- Frequency division multiplexing
 - divide frequency spectrum in *frequency bands*
 - separate frequency band per circuit



Definition: TDM

Divide time into **frames**, each frame into N **slots**; each circuit owns one slot per frame and uses the *full* link bandwidth during its slot. (Old telephone T1 lines: 24 voice channels, 8000 frames/s.)

Definition: FDM

Divide the frequency spectrum into N **bands**; each circuit owns one band *all the time*. (FM radio, old analog telephone trunks, cable TV channels.)

Formula: Per-circuit rate and transfer time

With N equal circuits on a link of rate R :

$$R_{\text{circuit}} = \frac{R}{N} \left[\frac{\text{bits}}{\text{s}} \right]$$

Derivation (TDM): in each frame of duration T_f , the link sends RT_f bits split among N slots; one circuit gets RT_f/N bits per T_f seconds $\Rightarrow$ rate R/N . (FDM: the bandwidth, hence capacity, is split N ways.)

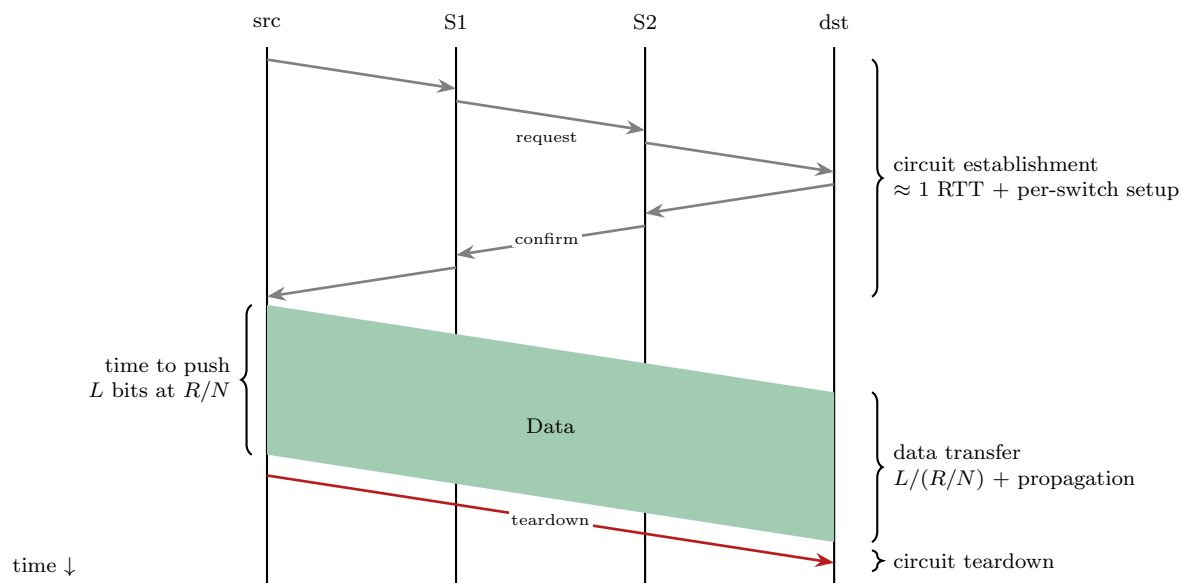
$$T_{\text{transfer}} = t_{\text{setup}} + \frac{L}{R/N} + d_{\text{prop}} \quad [\text{s}]$$

Common pitfall: TDM does not speed up bursts

Even if the other $N - 1$ circuits are idle, your circuit still gets only R/N . Idle slots are simply wasted — this is the precise mechanism behind “inefficient when traffic is bursty”.

1.4 Timing in circuit switching

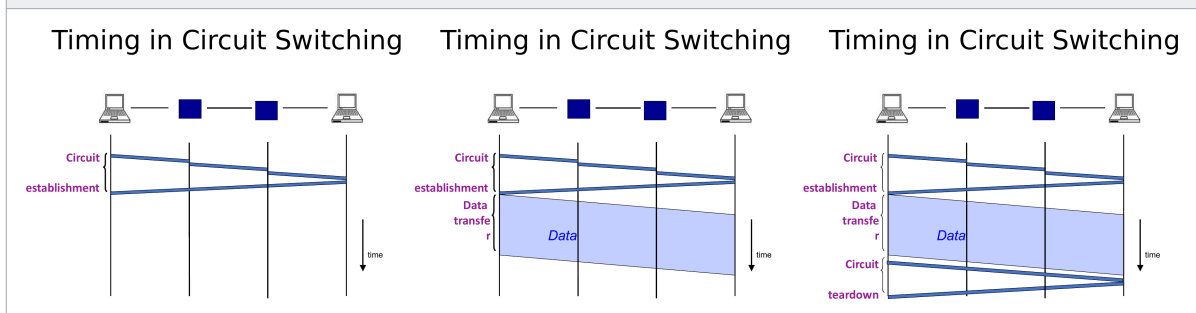
The slides animate this over ten frames; here is the complete picture.



Intuition: Reading the timing diagram

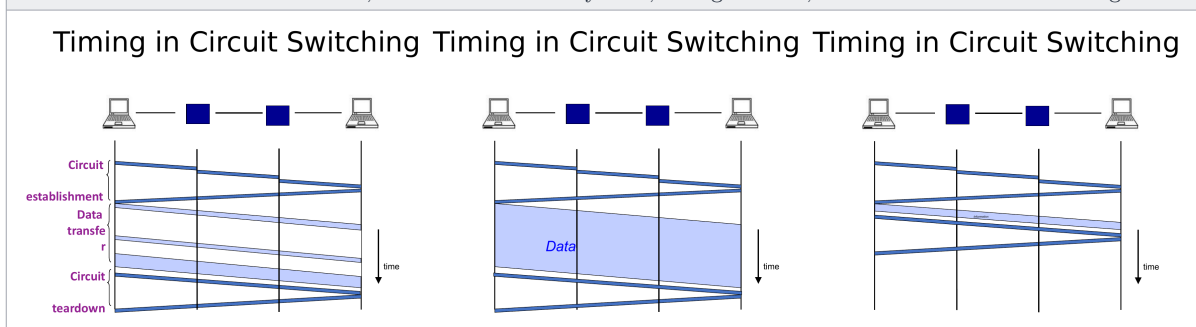
- The request hops switch by switch; each switch spends a little time deciding whether it can reserve capacity (the small vertical gap at each switch).
- Only after the confirmation returns can data start: a full **round trip of pure overhead**.
- The green data band is a *parallelogram*: its vertical height at src is the transmission time $L/(R/N)$; its slope is propagation. Notice the data does **not** pause at S1 or S2 — no store-and-forward, no queues. Bits flow straight through their reserved slots.
- Teardown frees the resources; until then they are held even if src is silent (e.g. a phone call with long pauses).

From the lecture slides: W2 L1, slides 9–11 — circuit establishment, data transfer, circuit teardown



1.4.1 Three timing scenarios on the slides

From the lecture slides: W2 L1, slides 12–14 — bursty data, a long transfer, a short “information” exchange



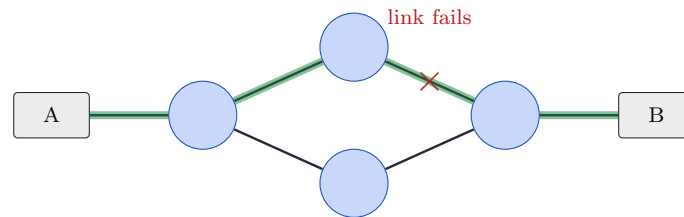
The last three timing frames are not decoration: each shows a different kind of traffic on the same circuit.

1. **Bursty data (slide 12).** The data arrives in several thin bands with gaps between them. During every gap the circuit is still reserved on every link, but carries nothing — this is exactly the “inefficient when traffic is bursty” con.
2. **A long transfer (slide 13).** The “Data” band is huge compared with the establishment and teardown zig-zags. The one-RTT setup cost is *amortised*: circuit switching works well here (and the data gets guaranteed rate, no queues).
3. **A short exchange (slide 14).** The “Information” band is a thin sliver; almost all the time goes on setting up and tearing down the circuit. For short messages (a web request, a DNS lookup) circuit switching is dominated by overhead.

Formula: Efficiency of a circuit

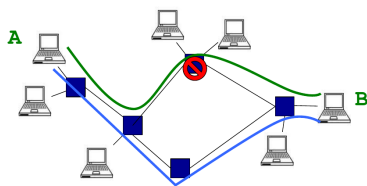
$$\eta = \frac{T_{\text{data}}}{T_{\text{setup}} + T_{\text{data}} + T_{\text{teardown}}} \quad [\text{dimensionless}]$$

Example: setup = 30 ms (one RTT), teardown = 15 ms. A 1 s transfer: $\eta = 1/1.045 = 96\%$.
 A 1 ms message: $\eta = 1/46 \approx 2\%$. Add idle gaps (bursty traffic) and the *reserved capacity* is wasted too, even when T_{data} is large.

1.5 Circuit switching doesn't "route around trouble"

an alternative path exists, but the circuit's state lives only in S1-S2-S4: the call drops and must be set up again

From the lecture slides: W2 L1, slides 15–16 — a failed switch breaks the circuit; pros and cons

Circuit Switching

Circuit switching doesn't "route around trouble"

Circuit Switching

- Pros
 - predictable performance
 - simple/fast switching (once circuit established)
- Cons
 - complexity of circuit setup/teardown
 - inefficient when traffic is bursty
 - circuit setup adds delay
 - switch fails → its circuit(s) fails

Intuition

The circuit *is* the state in specific switches. Lose one switch or link on it and the conversation is lost, even when the rest of the network could carry it. The original ARPANET motivation (survivable communication) was exactly to avoid this: in packet switching, the next packet just takes the other path.

1.6 Circuit switching: pros and cons**Pros**

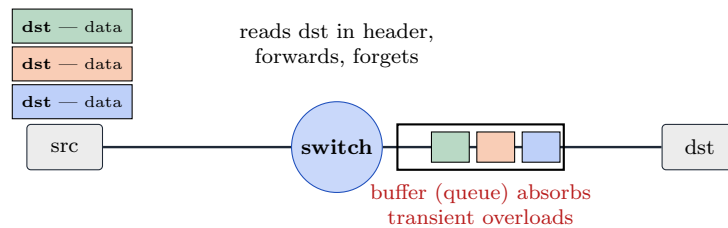
Predictable performance — guaranteed rate, constant delay, no loss from congestion
 Simple/fast switching once the circuit is established (slot-to-slot table)

Cons

Complexity of circuit setup/teardown (signalling protocol, per-switch state)
 Inefficient when traffic is bursty (reserved capacity idles)
 Circuit setup adds delay (at least one RTT before data)
 Switch fails ⇒ its circuit(s) fail (no rerouting)

2 Module 2.2 — Packet Switching

2.1 How packet switching works

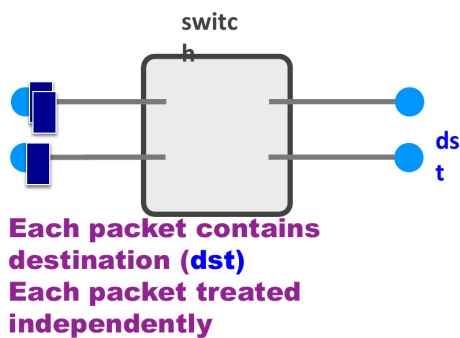


What the slide says (W2 L2, slides 1–2)

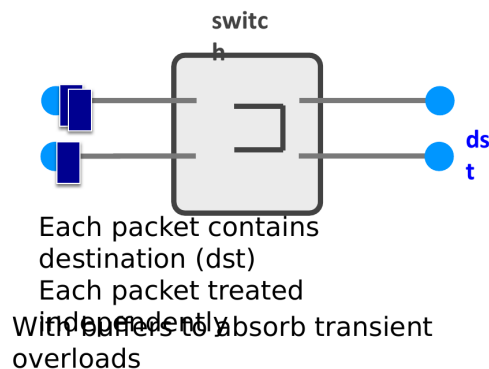
Each packet contains the destination (dst). Each packet is treated independently. With buffers to absorb transient overloads.

From the lecture slides: W2 L2, slides 1–2 — each packet carries dst; buffers absorb transient overloads

Packet switching



Packet switching



Definition: Packet switching

Data is split into packets, each carrying the full destination address in its header. Each switch examines every packet's header, looks up the output port in a **forwarding table**, and forwards it. No reservation, no per-conversation state: each packet is **independent** (a “datagram”).

Intuition: Couriers vs a reserved train

Circuit switching books the whole route in advance. Packet switching is like sending many independent courier parcels, each labelled with the address: at each hub a worker reads the label and throws it on the right truck. If a hub is busy, parcels wait on a shelf (the buffer). If one road closes, the next parcel is simply routed differently.

2.2 Packet switching: pros and cons

Pros	Cons
Efficient use of network resources (statistical multiplexing: idle capacity is used by whoever needs it)	Unpredictable performance (variable queuing delay, possible loss)
Simpler to implement (no setup/teardown signalling, no per-flow state in switches)	Requires buffer management and congestion control
Robust: can “route around trouble”	

2.3 Two approaches compared — and a hybrid

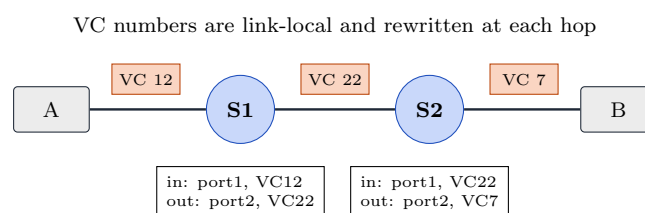
	Packet switching	Circuit switching
Resource allocation	packets treated on demand	resources reserved per active “connection”
Admission control	per packet (queue full $\Rightarrow$ drop)	per connection (no capacity $\Rightarrow$ block call)
State in switches	forwarding table only	per-circuit state
Header	full destination address	none needed (slot position identifies circuit)
Under overload	everyone slows down (delay, loss)	new calls rejected; admitted calls unaffected

Definition: Admission control

The decision of whether to accept new load into the network. Circuit switching decides once, when the connection is set up; packet switching decides for every individual packet (a full buffer means that packet is dropped).

Definition: Virtual circuit — the hybrid

Emulating circuit switching with packets. A setup phase picks a path and installs a small table entry in each switch; each packet then carries a short **virtual-circuit identifier (VCI)** instead of a full destination address. Switches forward by VCI lookup. Resources *may* be reserved per VC. Examples: X.25, Frame Relay, ATM, and today MPLS inside ISP networks.



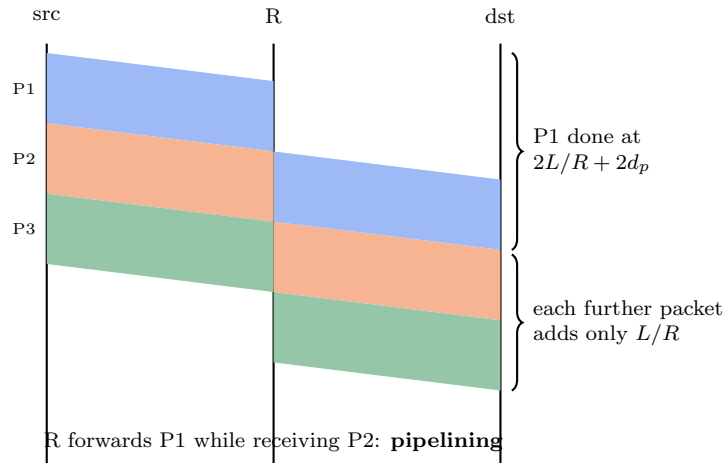
2.4 The Internet uses store-and-forward packet switching

What the slide says (W2 L2, slide 5)

The Internet uses store-and-forward packet switching.

Definition: Store-and-forward

A switch must receive the **entire** packet (and usually verify it) before it starts transmitting the packet on the outgoing link. So each hop contributes a full transmission time L/R .



Formula: End-to-end delay under store-and-forward (no queuing, negligible processing)

One packet, N links of equal rate R :

$$d = N \frac{L}{R} + \sum_{i=1}^N d_{\text{prop},i} \quad [\text{s}]$$

Derivation. The source spends L/R pushing the packet onto link 1; the last bit arrives at switch 1 after a further $d_{\text{prop},1}$. Only now can switch 1 start transmitting (store-and-forward): another $L/R + d_{\text{prop},2}$, and so on for N links.

P packets back-to-back:

$$d = (N + P - 1) \frac{L}{R} + \sum_i d_{\text{prop},i} \quad [\text{s}]$$

Derivation. The first packet arrives at $NL/R + \sum d_p$. Links work in parallel (a pipeline), so after the first packet, the bottleneck is simply the rate at which packets leave the last link: one every L/R . The remaining $P - 1$ packets add $(P - 1)L/R$.

Intuition: Why smaller packets reduce latency

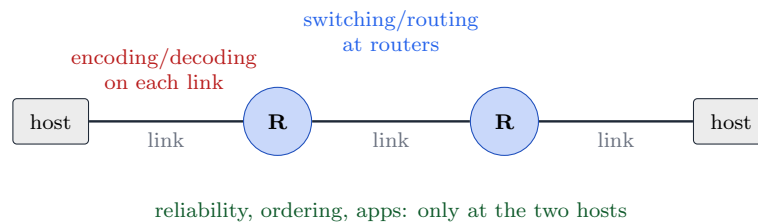
Send a message of M bits over N hops. As one packet: $d = NM/R$. As P packets of M/P bits: $d = (N + P - 1) \frac{M}{PR}$. For $N = 3$, $P = 10$: $\frac{12}{10} \frac{M}{R}$ vs $3 \frac{M}{R}$ — $2.5\times$ faster, because switches work in parallel. (Header overhead per packet limits how small you go.)

Beyond the slides: Cut-through switching

Some Ethernet switches start forwarding as soon as the destination field has arrived (“cut-through”), avoiding the per-hop L/R . It saves latency but may forward corrupted frames because the checksum at the end hasn’t been checked yet. The Internet’s routers are store-and-forward.

3 Module 2.3 — The Concept of Layering

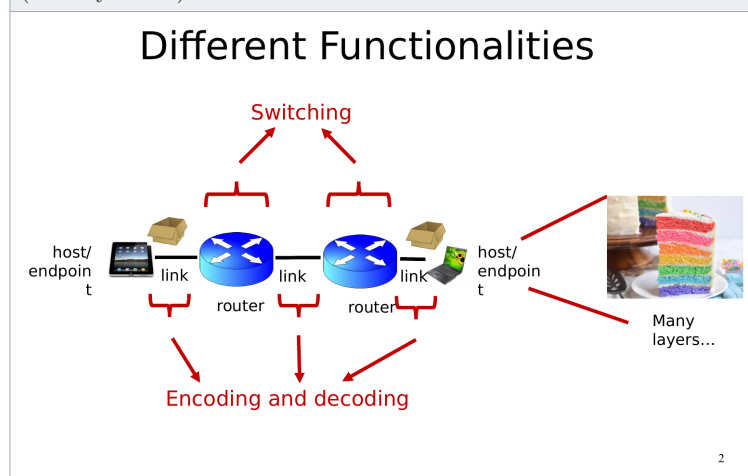
3.1 Different functionalities in different places



What the slide says (W2 L3, slide 2)

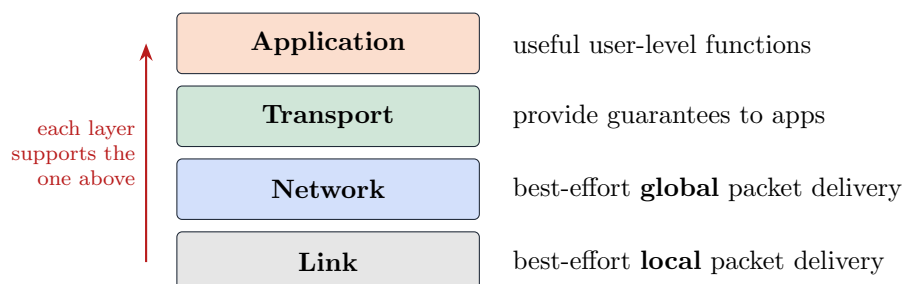
Encoding and decoding; switching; many layers... different functionalities.

From the lecture slides: W2 L3, slide 2 — where functionalities live (the “layer cake”)



Everything Week 1 listed — encoding, medium access, error detection, routing, reliability, congestion control, scheduling — has to live *somewhere*. Layering is the organising principle that decides where.

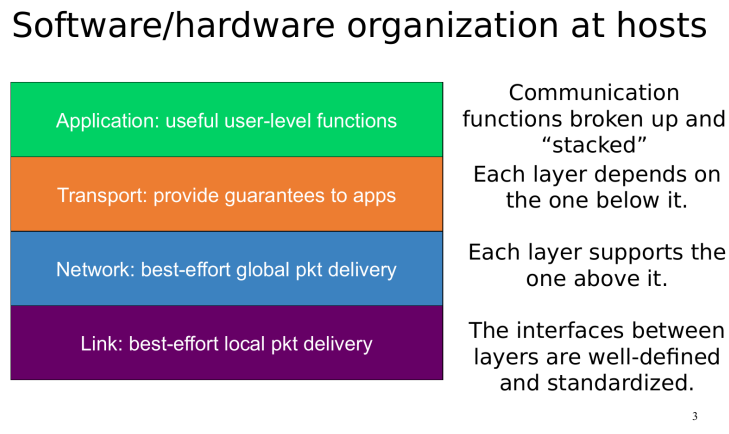
3.2 The layered stack



What the slide says (W2 L3, slide 3)

Software/hardware organisation at hosts. Communication functions broken up and “stacked”. Each layer depends on the one below it. Each layer supports the one above it. The interfaces between layers are well-defined and standardised.

From the lecture slides: W2 L3, slide 3 — the four-layer stack at hosts



Intuition: Local vs global

Link = get a frame to the *next device* on this one link (your laptop → WiFi router), best effort. **Network** = chain many link deliveries together to reach *any host on Earth*, still best effort. **Transport** = turn that unreliable host-to-host delivery into what the app needs (reliable byte stream, or just process-level delivery). **Application** = the actual job (fetch a web page, send mail).

Definition: Layering

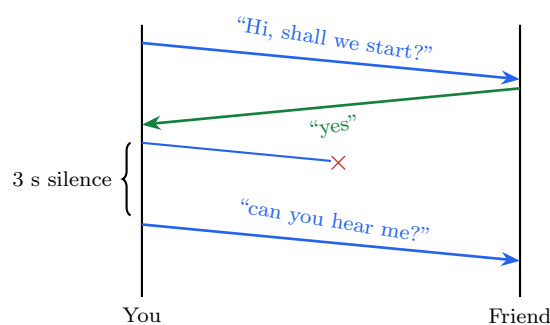
Internet software and hardware are arranged in **layers**. Each layer has a well-defined **function** and well-defined **interfaces** to the layers above and below it. Layering provides **modularity**. The functionality of each layer is implemented in **protocols**.

3.3 Protocols: the rules of networking

Definition: Protocol

A protocol consists of two things:

1. **Message format** — the structure of messages exchanged with an endpoint (fields, sizes, order, meaning).
2. **Actions** — operations upon receiving, *or not receiving*, messages.



Intuition: The Zoom example from the slide, decoded

Message format: English words and sentences — both sides must agree on the language, or the bits are meaningless. **Actions:** when a word is heard, say “yes” (an acknowledgement);

when nothing is heard for more than 3 seconds, say “can you hear me?” (a *timeout* followed by a retransmission/probe). This tiny example already contains ACKs and timeouts — the exact mechanisms TCP uses (Week 4). Note that a protocol must specify behaviour on *silence*; that’s where most bugs live.

3.4 The protocols of the Internet

What the slide says (W2 L3, slide 6)

Standardised by the **Internet Engineering Task Force (IETF)** through documents called **RFCs** (“Request For Comments”). Layering of protocols: FTP, HTTP, SIP, RTSP, HTTPS / TCP, UDP / IP / 802.11, X.25, ATM ...

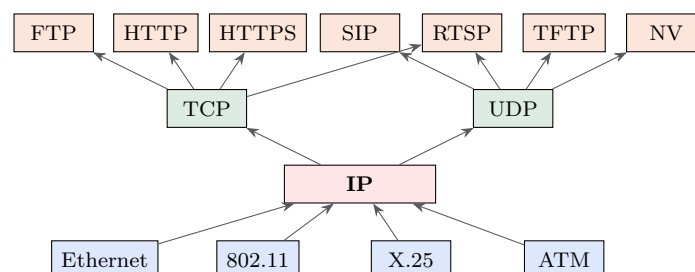
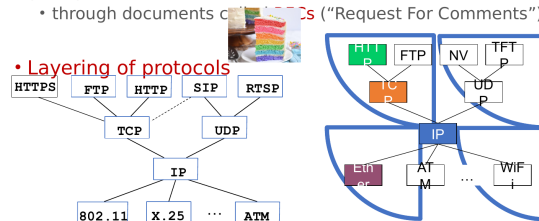
From the lecture slides: W2 L3, slides 5–6 — protocols = message format + actions; the Internet protocol graph

Protocols: The “rules” of networking

- Protocols consist of two things
 - **Message format**
 - structure of messages exchanged with an endpoint
 - **Actions**
 - operations upon receiving, or not receiving, messages
- **Example of a Zoom conversation:**
 - Message format: English words and sentences
 - Actions: when a word is heard, say “yes”; when nothing is heard for more than 3 seconds, say “can you hear me?”

The protocols of the Internet

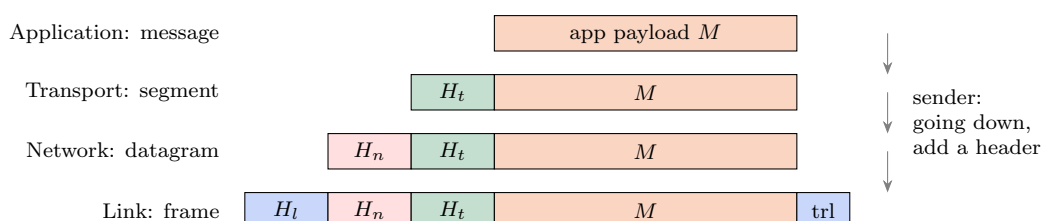
- Standardized by the Internet Engineering Task Force (IETF)
 - through documents called **RFCs** (“Request For Comments”)



Beyond the slides: How RFCs work

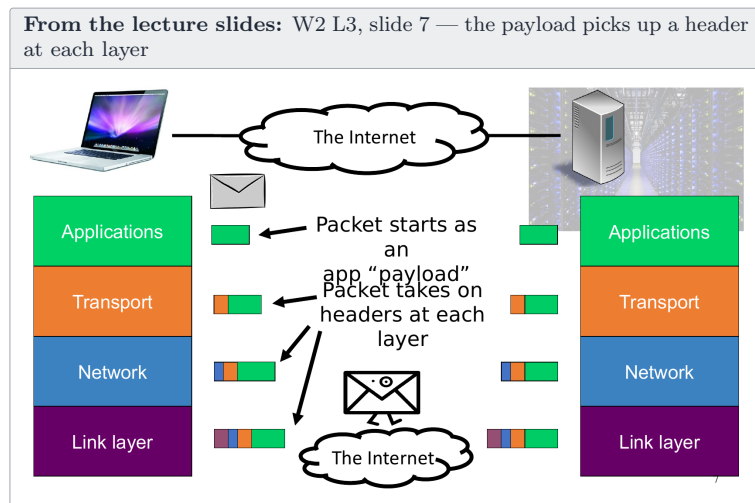
Anyone can write an Internet-Draft; working groups refine it by “rough consensus and running code”; approved drafts become numbered RFCs, which are never edited, only superseded. Examples: RFC 768 (UDP, 1980), RFC 791 (IPv4), RFC 793 → RFC 9293 (TCP), RFC 2616 → 9110 (HTTP semantics). Link layers like WiFi and Ethernet are standardised by the **IEEE** (802.x), not IETF.

3.5 Encapsulation: the packet takes on headers at each layer



What the slide says (W2 L3, slide 7)

Packet starts as an app “payload”; the packet takes on headers at each layer.

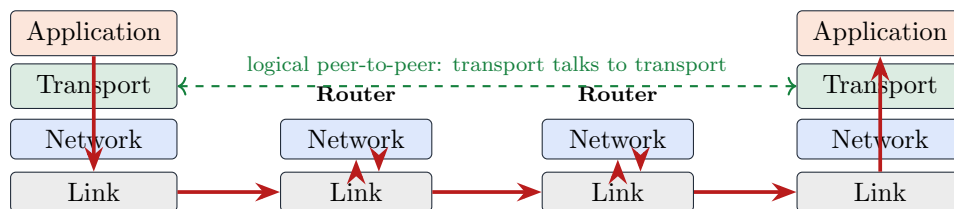


Layer	Unit (PDU) name	Header contains (examples)	Address used
Application	message	HTTP method, URL	domain names
Transport	segment (TCP) / datagram (UDP)	ports, seq/ACK numbers, checksum	port numbers
Network	datagram / packet	source & destination IP, TTL	IP addresses
Link	frame	source & destination MAC, type; trailer = CRC	MAC addresses

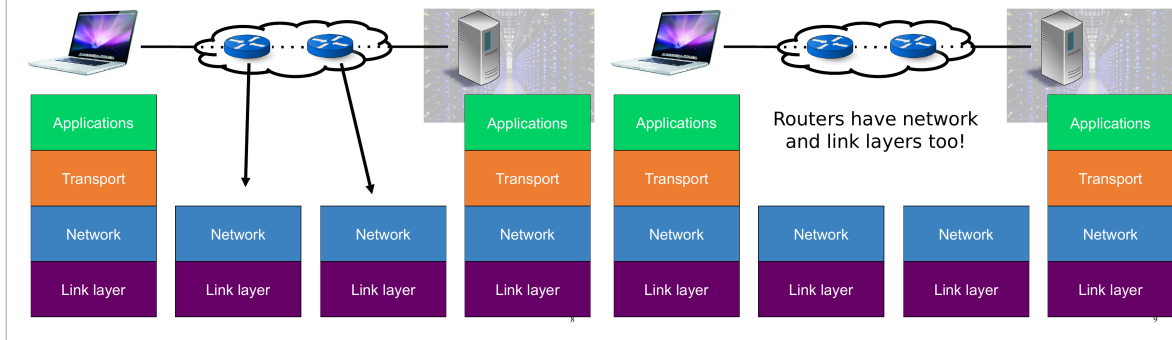
Intuition: Envelopes inside envelopes

You write a letter (message), put it in an envelope addressed to “Priya, Room 4” (transport header: which process), put that in a bigger envelope addressed to the building (network header: which host), and hand it to a courier with a local delivery slip (link header: next hop). Each handler reads only its own envelope. At the destination, each layer strips its own header (**decapsulation**) and passes the rest up.

3.6 Routers have network and link layers too!



From the lecture slides: W2 L3, slides 8–9 — routers process only network and link layers



Intuition

The data physically travels down the sender's stack, across links, *up to the network layer* at each router (to read the destination IP and pick an output), back down, and finally all the way up at the receiver. But *logically*, each layer talks only to its **peer** layer on the other end: the transport header written by host 1 is read only by host 2's transport. Routers don't need transport or application layers — this is the end-to-end principle made concrete, and it's why you can deploy a new application without touching a single router.

Beyond the slides: Which devices implement which layers

Device	Highest layer it processes	Decides using
Hub / repeater	Physical	nothing (copies bits to all ports)
Switch (Ethernet)	Link	MAC address
Router	Network	IP address
Host	Application	everything

(Middleboxes such as NATs and firewalls peek at transport headers too — a well-known violation of pure layering.)

4 Module 2.4 — Why Layering, and Measuring Networks

4.1 Why layering?

What the slide says (W2 L4, slide 1)

Communication over the Internet is a complex problem. **Separation of responsibilities:** layering simplifies understanding, testing, maintaining. **Modular, defined contract across layers:** easy to improve or replace the protocol at one layer without affecting others.

Intuition: Concrete wins of layering

- Your laptop switches from WiFi to Ethernet to 5G tethering mid-download — only the link layer changes; TCP and the browser don't notice (apart from maybe a new IP address).
- HTTP/1.1 → HTTP/2 → HTTP/3 changed the application protocol without replacing a single router.
- Debugging: if **ping** works but the website doesn't, the network layer is fine — look higher up.

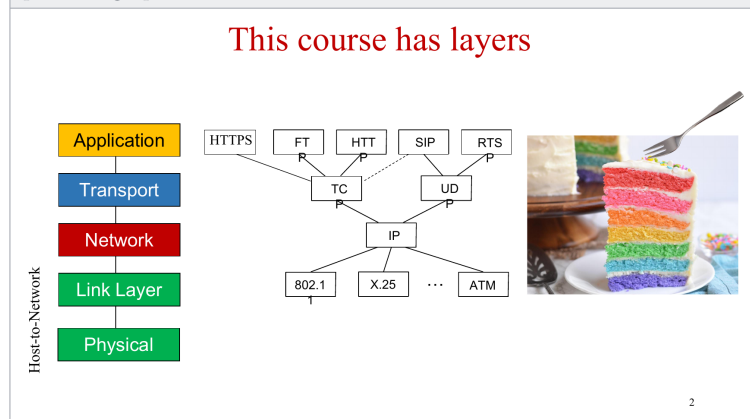
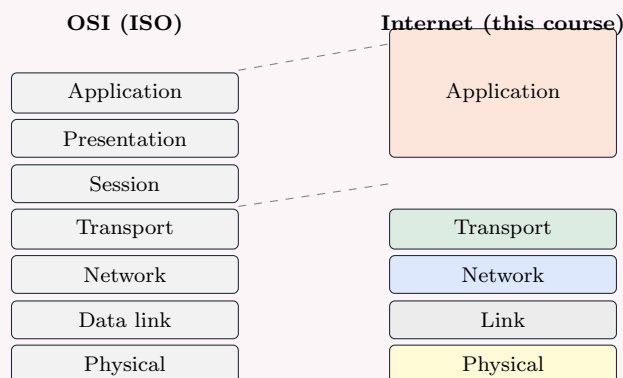
Common pitfall: Costs of layering

- (1) Header overhead: each layer adds bytes (TCP 20 + IP 20 + Ethernet 18 = 58 bytes around every payload). (2) Duplicated functionality (error checks at link *and* transport). (3) Information hiding hurts: TCP can't see that a loss was due to WiFi noise rather than congestion, so it slows down needlessly.

4.2 This course has layers

Application	FTP, HTTP, HTTPS, SIP, RTSP ...
Transport	TCP, UDP
Network	IP
Link (host-to-network)	802.11, Ethernet, X.25, ATM
Physical	copper, fibre, radio

From the lecture slides: W2 L4, slide 2 — the course's layers and the protocol graph

**Beyond the slides: OSI 7-layer vs Internet 5-layer**

The Internet folds presentation (encoding, encryption) and session (dialog control) into the application.

4.3 Measuring networks: definitions

What the slide says (W2 L4, slide 4)

Packet size: length of a packet (bits or bytes), incl. header and data. **Bandwidth:** for a single link, amount of data it can transmit per unit time (bits/s, Bytes/s or packets/s). **Propagation delay:** time needed to move one bit across; imposed by the medium; depends on the link “length”. **Transmission delay:** time from first bit@sender to last bit@sender; determined by link bandwidth and packet size. **Queueing delay:** time a packet waits for transmission; determined by contention for the link. **Total packet delay:** time from first bit@sender to last bit@receiver = propagation + queueing + transmission delay (for a single packet).

Formula: Total single-hop packet delay

$$d_{\text{total}} = d_{\text{trans}} + d_{\text{prop}} + d_{\text{queue}} (+ d_{\text{proc}}) = \frac{L}{R} + \frac{D}{s} + d_{\text{queue}} \quad [\text{s}]$$

Common pitfall: Precise start and end points

Transmission delay is measured *at the sender*: first bit leaves → last bit leaves. Total delay is first bit leaves the sender → last bit arrives at the receiver. That’s why the total is transmission + propagation, not propagation alone.

5 Module 2.5 — The Four Components of Delay

5.1 1. Propagation delay

Formula: Propagation delay

How long does it take one bit to move from one end of a link to the other?

$$d_{\text{prop}} = \frac{\text{link length}}{\text{propagation speed of link}} \quad \left[\text{s} = \frac{\text{m}}{\text{m/s}} \right]$$

Slide example: $\frac{30 \text{ km}}{3 \times 10^8 \text{ m/s}} = \frac{3 \times 10^4}{3 \times 10^8} = 10^{-4} \text{ s} = 0.1 \text{ ms}.$

Independent of packet size and of bandwidth. The only ways to reduce it: shorter paths or faster media (e.g. microwave links in air, $\approx c$, are used by high-frequency traders instead of fibre, $\approx 2c/3$).

5.2 Bandwidth-delay product

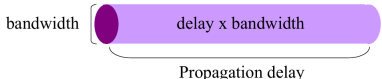
Formula: BDP

$$\text{BDP} = \text{bandwidth} \times \text{propagation delay} \quad \left[\text{bits} = \frac{\text{bits}}{\text{s}} \times \text{s} \right]$$

= the number of bits “in flight” on the link at any time. **Derivation.** A bit takes d_{prop} to cross the link. During that time the sender (transmitting continuously) emits $R \times d_{\text{prop}}$ further bits. So when the first bit arrives, exactly $R d_{\text{prop}}$ bits are on the wire.

From the lecture slides: W2 L5, slides 2–3 — the BDP pipe and the two examples

Concept of Bandwidth Delay Product

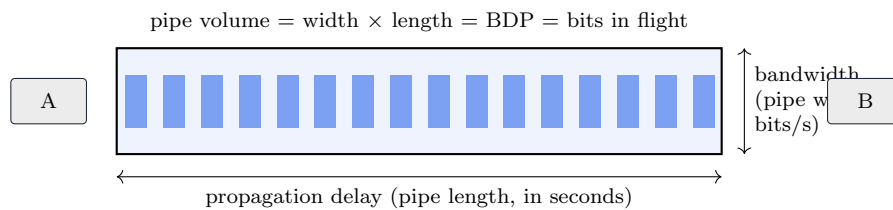
- 
- Link bandwidth
 - number of bits sent/received per unit time (bits/sec or bps)
 - Propagation delay
 - time for one bit to move through the link (seconds)
 - Bandwidth-Delay Product (BDP)
 - number of bits “in flight” at any time
 - BDP = bandwidth × propagation delay

Examples

- Same city over a slow link:
 - bandwidth: ~100Mbps
 - propagation delay: ~0.1msec
 - BDP: 10,000bits (1.25KBytes)
- Cross-country over fast link:
 - bandwidth: ~10Gbps
 - propagation delay: ~10msec
 - BDP: 10⁸bits (12.5MBytes)

2

3



Slide examples:

Scenario	Bandwidth	Prop. delay	BDP (bits)	BDP (bytes)
Same city, slow link	100 Mbps	0.1 ms	$10^8 \times 10^{-4} = 10^4$	1.25 KB
Cross-country, fast link	10 Gbps	10 ms	$10^{10} \times 10^{-2} = 10^8$	12.5 MB

Intuition: Why BDP matters

BDP is how much data you must “pour in” before anything comes out the other end. A protocol that waits for a reply before sending more (Week 4’s stop-and-wait) can only have one packet in the pipe; on the cross-country link, one 1500-byte packet fills $12,000/10^8 = 0.012\%$ of the pipe. To use the link fully, a sender needs $\approx$ BDP (actually bandwidth $\times$ RTT, the round-trip pipe) of unacknowledged data outstanding — the whole motivation for sliding windows.

5.3 2. Transmission delay

Formula: Transmission delay

How long does it take to push all the bits of a packet into a link?

$$d_{\text{trans}} = \frac{\text{packet size}}{\text{transmission rate of link}} = \frac{L}{R} \quad \left[s = \frac{\text{bits}}{\text{bits/s}} \right]$$

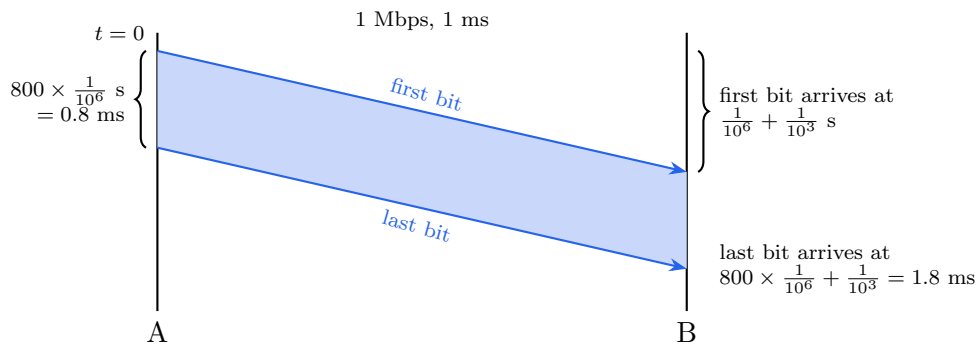
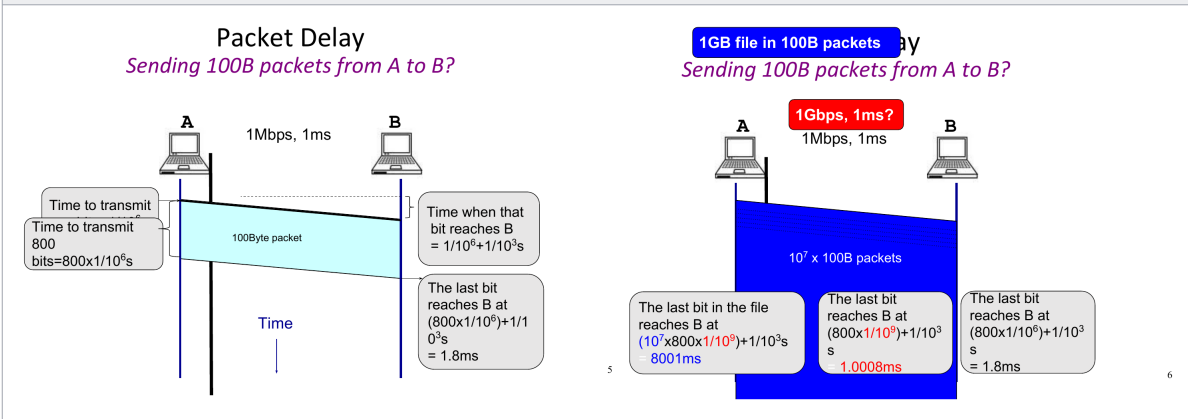
Slide example: $\frac{1000 \text{ bits}}{100 \text{ Mbps}} = \frac{10^3}{10^8} = 10^{-5} \text{ s} = 10 \mu\text{s}.$

5.4 Packet delay: the slide's worked example

What the slide says (W2 L5, slides 5–6)

Sending 100 B packets from A to B over a link of 1 Mbps and 1 ms propagation.

From the lecture slides: W2 L5, slides 5–6 — one 100 B packet at 1 Mbps; then 1 Gbps and a 1 GB file



1. Packet size $L = 100 \text{ B} = 800 \text{ bits}$. Time to transmit one bit = $1/R = 1/10^6 \text{ s} = 1 \mu\text{s}$.
2. Time to transmit 800 bits = $800 \times 10^{-6} = 0.8 \text{ ms}$.
3. The last bit leaves A at 0.8 ms and needs 1 ms to propagate: arrives at $0.8 + 1 = 1.8 \text{ ms}$.

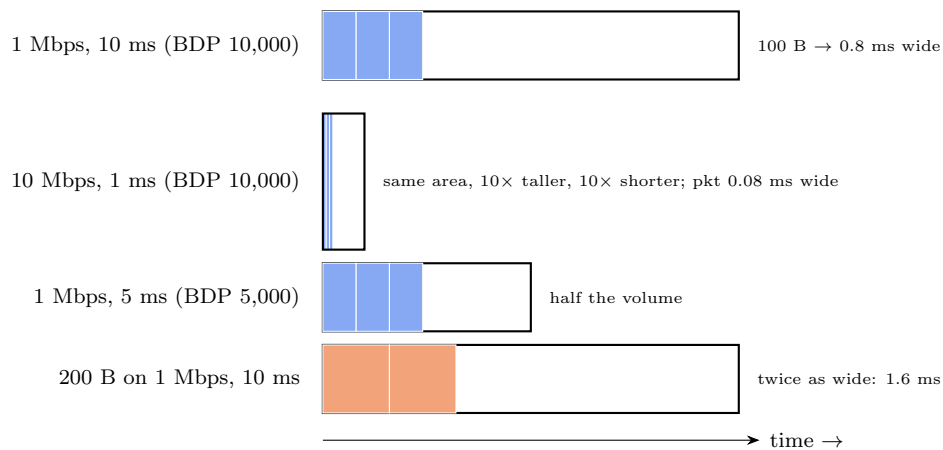
Case	d_{trans}	d_{prop}	Last bit at B
100 B, 1 Mbps, 1 ms	$800/10^6 = 0.8 \text{ ms}$	1 ms	1.8 ms
100 B, 1 Gbps, 1 ms	$800/10^9 = 0.8 \mu\text{s}$	1 ms	1.0008 ms
1 GB file in 100 B pkts, 1 Gbps, 1 ms	$10^7 \times 800/10^9 = 8 \text{ s}$	1 ms	8001 ms

Intuition: Which delay dominates?

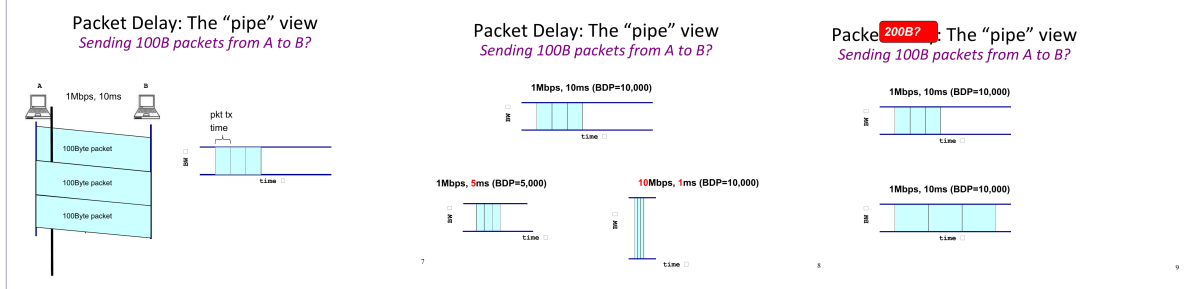
A 1000× faster link barely helps a single small packet ($1.8 \rightarrow 1.0008 \text{ ms}$) because propagation dominates — you can't buy your way past the speed of light. For a 1 GB file, the 8 s of transmission dwarfs 1 ms of propagation — there, bandwidth is everything. **Latency-bound** vs **bandwidth-bound** is the first question to ask about any transfer.

5.5 Packet delay: the “pipe” view

Plot bandwidth vertically and time horizontally. A packet becomes a rectangle of height R (bits/s) and width L/R (s), so its **area is L bits** whatever the link. The pipe has length d_{prop} and height R : **area = BDP**.



From the lecture slides: W2 L5, slides 7–9 — pipe view: 1 Mbps/10 ms, 1 Mbps/5 ms, 10 Mbps/1 ms, and 200 B packets



Intuition

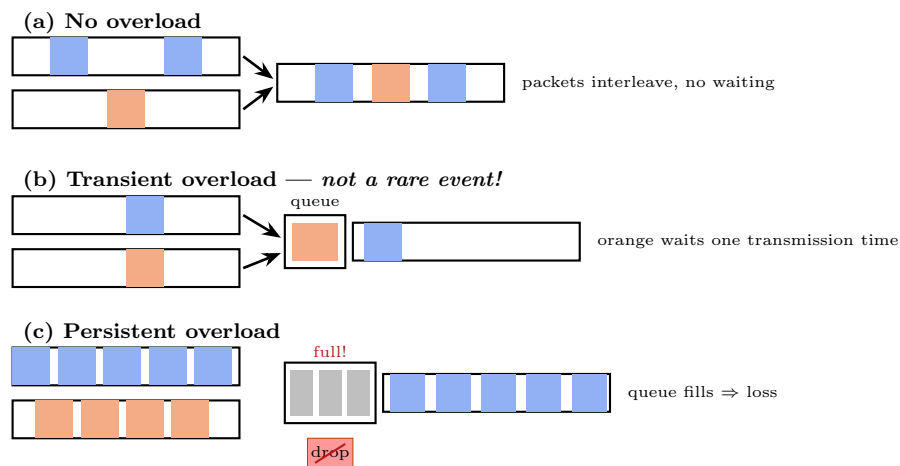
How many 100-byte packets “fit” in the pipe? $\text{BDP} / \text{packet size} = 10,000 / 800 = 12.5$ in the first two pipes, 6.25 in the third. Two links with the same BDP hold the same amount of data in flight even though one is “fast and short” and the other “slow and long”. This view makes it obvious why doubling the packet size doubles its width (transmission time) but does nothing to the pipe’s length (propagation).

5.6 3. Queuing delay

Formula: Queuing delay

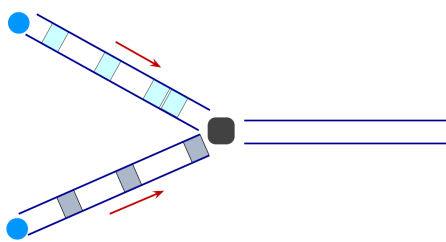
How long does a packet have to sit in a buffer before it is processed (transmitted)?

5.6.1 The pipe view of queuing (slides 13–20)

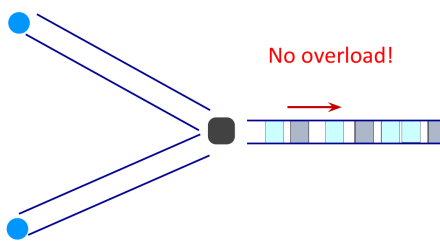


From the lecture slides: W2 L5, slides 12–15 — no overload, then transient overload builds a queue

Queuing delay: “pipe” view

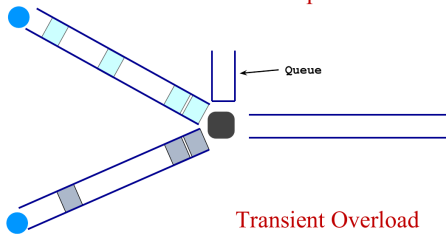


Queuing delay: “pipe” view

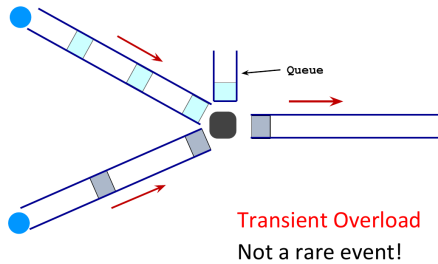


Queuing delay: “pipe” view

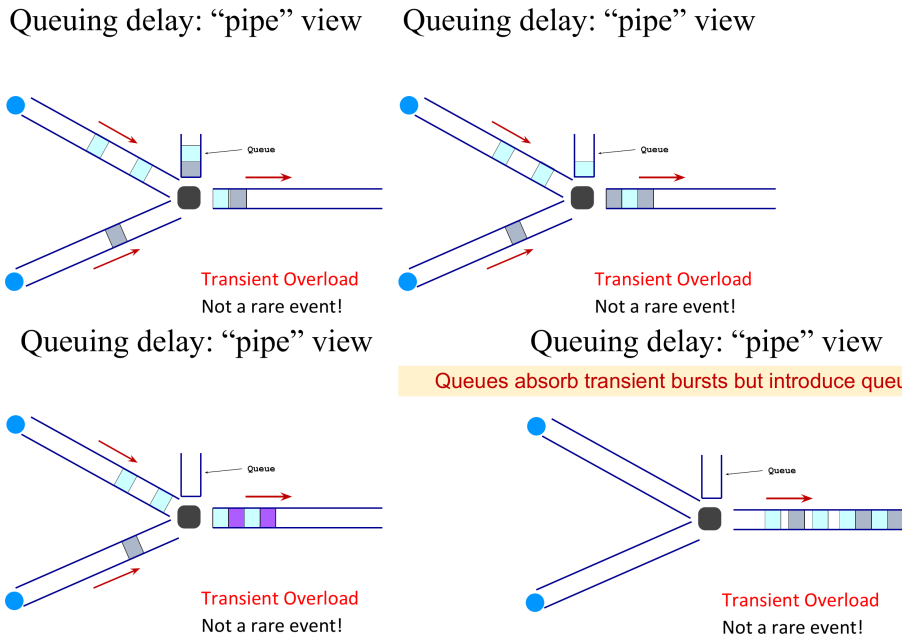
The Internet uses store-and-forward packet switching.



Queuing delay: “pipe” view



From the lecture slides: W2 L5, slides 16–19 — the queue drains; persistent overload ends in loss



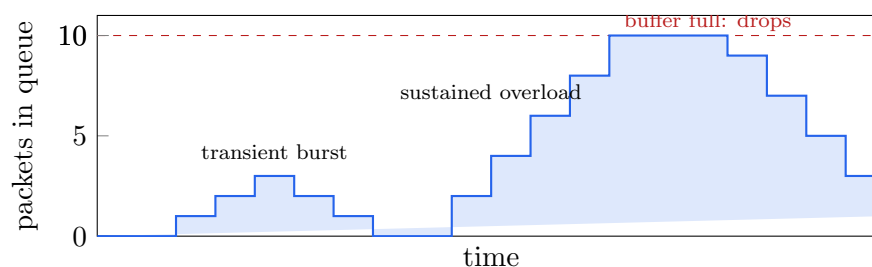
What the slide says (W2 L5, slides 14–22)

No overload → no queue. **Transient overload** — **not a rare event!** Queues absorb transient bursts *but introduce queuing delay*. The Internet uses store-and-forward packet switching. What about **persistent overload**? Will eventually **drop packets** (“loss”).

Intuition: Why transient overload is “not rare”

Whenever two packets from two input links arrive at the same time for the same output link, one must wait — even if both inputs are lightly loaded on average. With many inputs and bursty traffic (statistical multiplexing!), such collisions happen all the time. Queues are the price of on-demand sharing.

5.6.2 Queue length over time



5.6.3 What queuing delay depends on

What the slide says (W2 L5, slides 23–24)

Depends on the traffic pattern: **arrival rate** at the queue; **nature of arriving traffic** (bursty or not?); **transmission rate of outgoing link**. Characterised with statistical measures: **average** queuing delay; **variance** of queuing delay; **probability delay exceeds a threshold**

value.

Intuition

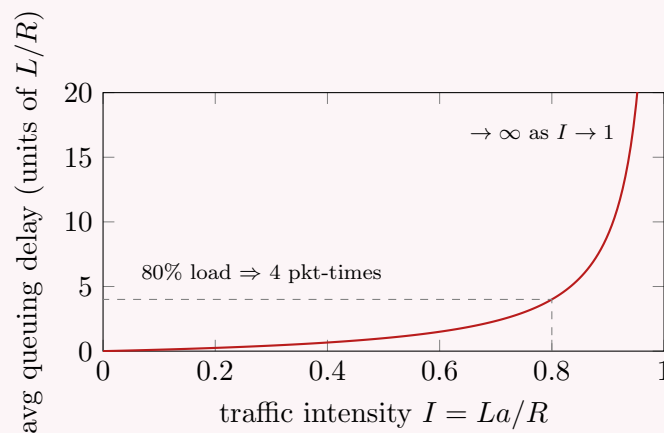
Unlike propagation and transmission (fixed numbers for a given link and packet), queuing delay is a *random variable* — it changes from packet to packet. The average tells you typical performance, the variance tells you **jitter** (what hurts video calls), and the tail probability ($P[\text{delay} > 150 \text{ ms}]$) tells you how often a real-time app will break.

Beyond the slides: Traffic intensity and the delay curve

Let a = average packet arrival rate (packets/s), L = packet size (bits), R = link rate. Then

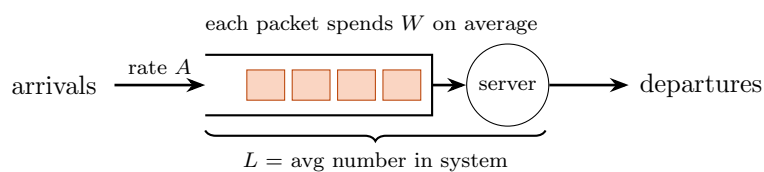
$$I = \frac{La}{R} \quad \left[\frac{\text{bits/pkt} \cdot \text{pkt/s}}{\text{bits/s}} = \text{dimensionless} \right]$$

is the fraction of link capacity demanded. $I > 1$: persistent overload, queue grows without bound. $I \leq 1$: depends on burstiness; for random (Poisson) arrivals the average queuing delay grows like $\frac{I}{1-I} \cdot \frac{L}{R}$ — it explodes as $I \rightarrow 1$.



Design rule: never run a link near 100% utilisation if you care about delay.

5.7 Basic queuing theory terminology



- **Arrival process** — how packets arrive: average rate A , peak rate P .
- W — average time packets wait in the queue/system (W for “waiting time”).
- L — average number of packets waiting in the queue/system (L for “length of queue”).

Common pitfall: Two different L 's!

In delay formulas L is *packet length in bits*; in Little's law L is *queue length in packets*. Read the context.

5.8 Little's Law (John Little, 1961)

Formula: Little's Law

$$L = A \times W \quad \left[\text{packets} = \frac{\text{packets}}{s} \times s \right]$$

The long-term average number L of customers in a stationary system equals the long-term average effective arrival rate A multiplied by the average time W a customer spends in the system.

What the slide says (W2 L5, slide 24)

The relationship is **not influenced** by the arrival process distribution, the service distribution, the service order, or practically anything else. In most queuing systems, service time is the bottleneck that creates the queue.

From the lecture slides: W2 L5, slides 23–25 — queuing terminology, Little's law, the service-counter example

Basic Queuing Theory Terminology

- Arrival process: how packets arrive
 - Average rate A
 - Peak rate P
- W : average time packets wait in the queue
 - W for "waiting time"
- L : average number of packets waiting in the queue
 - L for "length of queue"

Little's Law (John Little, 1961)

$$L = A \times W$$

The long-term average number L of customers in a stationary system is equal to the long-term average effective arrival rate A multiplied by the average time W that a customer spends in the system.

The relationship is **not influenced** by the arrival process distribution, the service distribution, the service order, or practically anything else. In most queuing systems, service time is the bottleneck that creates the queue.

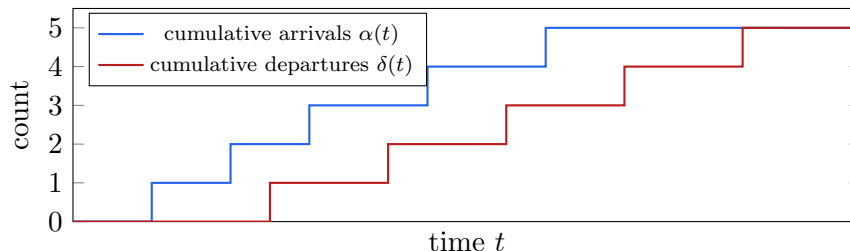
Little's Law (John Little, 1961)

$$L = A \times W$$

At a customer service counter, an operator has on average 4 customers and on an average 2 customers arrive every 3 minutes. What is the average waiting time of a customer?

Length of queue (L) = 4
 Arrival rate (A) = 2/3 customers per minute
 Waiting time (W) = 4 customers / (2/3 customers/minute)
 = 6 minutes

5.8.1 Why Little's Law is true — a derivation



1. At any time t , the number in the system is $N(t) = \alpha(t) - \delta(t)$ — the vertical gap between the curves.
2. Each customer i contributes a horizontal strip of length W_i (its time in the system) between the curves.
3. Over a long interval $[0, T]$ the area between the curves is computed two ways:

$$\int_0^T N(t) dt = \sum_{i=1}^{\alpha(T)} W_i.$$

4. Divide by T and rewrite the right side:

$$\underbrace{\frac{1}{T} \int_0^T N(t) dt}_{\rightarrow L} = \underbrace{\frac{\alpha(T)}{T}}_{\rightarrow A} \cdot \underbrace{\frac{1}{\alpha(T)} \sum_i W_i}_{\rightarrow W}.$$

5. Letting $T \rightarrow \infty$ in a stationary system gives $L = AW$. Nothing about distributions or order was used.

Intuition: The toll-payment view

Charge every customer Rs. 1 per second while inside. The shop collects L rupees per second on average (one per customer present). Equivalently, A customers arrive per second and each eventually pays W rupees: AW per second. Same money $\Rightarrow L = AW$.

5.8.2 Slide example

At a customer service counter, an operator has on average 4 customers, and on average 2 customers arrive every 3 minutes.

$$L = 4, \quad A = \frac{2}{3} \text{ customers/min} \quad \Rightarrow \quad W = \frac{L}{A} = \frac{4}{2/3} = \mathbf{6 \text{ minutes.}}$$

5.8.3 Applying Little's law to networks — which “system”?

You can draw the system boundary anywhere, as long as L , A , W refer to the *same* boundary:

System boundary	W is	L is
Just the queue	queuing delay	avg packets waiting
Queue + transmitter	queuing + transmission	avg packets in router
Whole network path	one-way delay	avg packets in flight

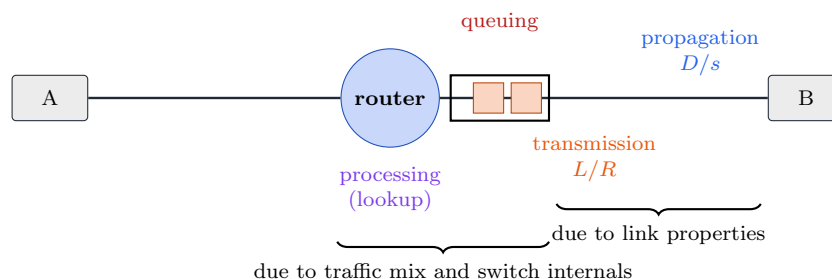
Applied to the whole path: packets in flight = rate $\times$ delay — which is exactly the bandwidth-delay product! BDP is Little's law for a pipe.

5.9 4. Processing delay

What the slide says (W2 L5, slide 29)

How long does the switch take to process a packet? Typically assume this is negligible.

Processing = checking header integrity, looking up the forwarding table, deciding the output port. Modern routers do this in nanoseconds; software routers/firewalls can take microseconds.

5.10 Recap on delay

Summary: The four delays

Delay	Formula	Depends on	Typical
Transmission	L/R	packet size, link rate	μs – ms
Propagation	D/s	distance, medium	μs – 100 ms
Queuing	random	load, burstiness, link rate	0– 100s of ms
Processing	—	router hardware	ns – μs

6 Formula Sheet

Quantity	Formula	Units
Circuit rate (TDM/FDM)	R/N	bits/s
Circuit transfer time	$t_{\text{setup}} + L/(R/N) + d_{\text{prop}}$	s
Store-and-forward, 1 pkt, N links	$N L/R + \sum d_{\text{prop}}$	s
Store-and-forward, P pkts	$(N + P - 1) L/R + \sum d_{\text{prop}}$	s
Propagation delay	D/s	s
Transmission delay	L/R	s
Total single-hop delay	$L/R + D/s + d_{\text{queue}} + d_{\text{proc}}$	s
Bandwidth-delay product	$R \times d_{\text{prop}}$	bits
Packets in the pipe	BDP / L	packets
Traffic intensity	$I = La/R$ (< 1 required)	—
Little's law	$L = AW$	packets

7 Numerical Problems with Worked Solutions

Problem 1. *Little's Law — Exercise 1 (slide)*

A network router processes packets at an average arrival rate of 200 packets per second. The average time a packet spends in the system (queue + processing) is 0.01 s. Calculate the average number of packets in the router's buffer.

Solution

$$L = AW = 200 \text{ pkt/s} \times 0.01 \text{ s} = \mathbf{2} \text{ packets.}$$

Problem 2. *Little's Law — Exercise 2 (slide)*

A network switch has an average of 10 packets in the system, and each packet spends an average of 0.05 s in the system. Determine the arrival rate.

Solution

$$A = L/W = 10/0.05 = \mathbf{200} \text{ packets/s.}$$

Problem 3. *Little's Law — Exercise 3 (slide)*

Packets of 1000 bits are sent from Chennai to Kolkata. The one-way delay varies between 50 ms (no queueing) and 125 ms (full queue), with an average of 75 ms. The sender transmits at 1 Mbit/s; the receiver gets packets at the same rate without loss. What is the average length of the queue at the bottleneck link (queueing happens only in one switch)? What is the buffer size?

Solution

Step 1 — arrival rate. $A = \frac{10^6 \text{ bits/s}}{1000 \text{ bits/pkt}} = 1000 \text{ packets/s}$ (no loss, so arrival = departure).

Step 2 — pick the system = the queue. The 50 ms with an empty queue is everything *except* queueing. So average queueing delay $W = 75 - 50 = 25 \text{ ms}$.

Step 3 — Little. $L = AW = 1000 \times 0.025 = \mathbf{25 \text{ packets}}$.

Buffer size. Full queue adds $125 - 50 = 75 \text{ ms}$. A full queue drains at 1000 pkt/s , so it holds $1000 \times 0.075 = \mathbf{75 \text{ packets}} = 75,000 \text{ bits}$.

Problem 4. Propagation (slide)

Compute the propagation delay of a 30 km link at $3 \times 10^8 \text{ m/s}$, and of a 3000 km fibre at $2 \times 10^8 \text{ m/s}$.

Solution

$$3 \times 10^4 / 3 \times 10^8 = 10^{-4} \text{ s} = 0.1 \text{ ms.} \quad 3 \times 10^6 / 2 \times 10^8 = 1.5 \times 10^{-2} \text{ s} = 15 \text{ ms.}$$

Problem 5. Transmission (slide)

Transmission delay of a 1000-bit packet at 100 Mbps; of a 1500-byte packet at 10 Mbps and at 1 Gbps.

Solution

$$10^3 / 10^8 = 10 \text{ } \mu\text{s.} \quad 12000 / 10^7 = 1.2 \text{ ms.} \quad 12000 / 10^9 = 12 \text{ } \mu\text{s.}$$

Problem 6. Packet delay (slide)

100-byte packets from A to B. (a) 1 Mbps, 1 ms. (b) 1 Gbps, 1 ms. (c) A 1 GB file in 100 B packets over 1 Gbps, 1 ms.

Solution

(a) $800 / 10^6 + 10^{-3} = 0.8 + 1 = \mathbf{1.8 \text{ ms}}$. (b) $800 / 10^9 + 10^{-3} = \mathbf{1.0008 \text{ ms}}$. (c) $10^9 / 100 = 10^7$ packets, sent back to back: last bit at $10^7 \times 800 / 10^9 + 10^{-3} = 8 + 0.001 = \mathbf{8.001 \text{ s}}$.

Problem 7. BDP (slide)

Compute the BDP for: (a) 100 Mbps, 0.1 ms; (b) 10 Gbps, 10 ms; (c) 1 Mbps, 10 ms; (d) 10 Mbps, 1 ms; (e) 1 Mbps, 5 ms. How many 100-byte packets fit in each of (c)–(e)?

Solution

(a) $10^4 \text{ bits} = 1.25 \text{ KB}$. (b) $10^8 \text{ bits} = 12.5 \text{ MB}$. (c) $10^6 \times 10^{-2} = 10^4 \text{ bits} \Rightarrow 10^4 / 800 = 12.5 \text{ pkts}$. (d) $10^7 \times 10^{-3} = 10^4 \text{ bits} \Rightarrow 12.5 \text{ pkts}$ (same pipe volume as (c)). (e) $5000 \text{ bits} \Rightarrow 6.25 \text{ pkts}$.

Problem 8. Circuit switching with TDM

A file of 640,000 bits is sent over a circuit-switched network. Every link uses TDM with 24 slots and has rate 1.536 Mbps. Circuit setup takes 500 ms. How long until the file is fully received (ignore propagation)?

Solution

Circuit rate = $1.536 \times 10^6 / 24 = 64$ kbps. Transfer = $640,000 / 64,000 = 10$ s. Total = $0.5 + 10 = 10.5$ s.

Problem 9. *Circuit vs packet switching*

A 10 Mbit file crosses 3 links, each 10 Mbps with 5 ms propagation. (a) Circuit switching: 4 TDM slots per link; setup takes one RTT. (b) Packet switching: 10,000-bit packets, idle network, store-and-forward.

Solution

(a) $RTT = 2 \times 3 \times 5 = 30$ ms. Circuit rate = 2.5 Mbps; transfer = $10^7 / 2.5 \times 10^6 = 4$ s; plus 15 ms propagation. Total $\approx 0.030 + 4 + 0.015 = 4.045$ s.

(b) $P = 10^7 / 10^4 = 1000$ packets, $L/R = 10^4 / 10^7 = 1$ ms, $N = 3$. $d = (N + P - 1)L/R + 3 \times 5$ ms = $1002 + 15 = 1017$ ms.

Packet switching wins $\approx 4\times$: it uses the full 10 Mbps (no idle slots) and has no setup.

Problem 10. *Why packetise?*

A 1 Mbit message crosses 4 store-and-forward links of 10 Mbps each (ignore propagation). Compare sending it as (a) one packet, (b) 100 packets of 10 kbit.

Solution

(a) $4 \times 10^6 / 10^7 = 0.4$ s. (b) $L/R = 10^4 / 10^7 = 1$ ms; $(4 + 100 - 1) \times 1 = 103$ ms. Pipelining across the four links makes it almost $4\times$ faster.

Problem 11. *Total delay across a router*

A 1500-byte packet goes $A \rightarrow R \rightarrow B$. Link 1: 100 Mbps, 200 km. Link 2: 10 Mbps, 800 km. $s = 2 \times 10^8$ m/s. At R the packet finds 3 identical packets ahead of it in the output queue; processing is $20 \mu\text{s}$. Find the total delay.

Solution

Link 1: $d_{\text{trans}} = 12000 / 10^8 = 0.12$ ms; $d_{\text{prop}} = 2 \times 10^5 / 2 \times 10^8 = 1$ ms.

At R: processing 0.02 ms; queuing = $3 \times 12000 / 10^7 = 3.6$ ms.

Link 2: $d_{\text{trans}} = 12000 / 10^7 = 1.2$ ms; $d_{\text{prop}} = 8 \times 10^5 / 2 \times 10^8 = 4$ ms.

Total = $0.12 + 1 + 0.02 + 3.6 + 1.2 + 4 = 9.94$ ms.

Problem 12. *Traffic intensity*

Packets of 1250 bytes arrive at a router output at 800 packets/s. The link is 10 Mbps. Compute the traffic intensity and say what happens. What if the link were 8 Mbps?

Solution

$La = 10,000 \times 800 = 8 \times 10^6$ bits/s. On 10 Mbps: $I = 0.8$ — stable but with substantial queuing delay (about 4 packet-times for random arrivals). On 8 Mbps: $I = 1$ — the queue has no spare capacity to drain after any burst; delay grows without bound and packets will be dropped.

Problem 13. *Little's law on a whole path*

A TCP connection keeps on average 50 packets in flight and its one-way delay averages 40 ms. What is its throughput in packets/s, and in Mbps for 1500-byte packets?

Solution

$A = L/W = 50/0.04 = 1250 \text{ pkt/s} \Rightarrow 1250 \times 12000 = 15 \text{ Mbps}$. (This is the idea behind “throughput = window / RTT”.)

8 Multiple-Choice Questions with Explanations

Q1. The correct order of steps in circuit switching is:

- (a) Send data, reserve, teardown
- (b) Reserve, establish circuit, send data, teardown
- (c) Establish, teardown, send data
- (d) Send data, establish, reserve

(b). Request $\rightarrow$ switches establish $\rightarrow$ data $\rightarrow$ teardown.

Q2. In TDM, each circuit gets:

- (a) A fixed frequency band all the time
- (b) A recurring time slot using the full link rate
- (c) A dedicated cable
- (d) The full link whenever it wants

(b). FDM gives a frequency band.

Q3. With N TDM circuits on a link of rate R , each circuit's rate is:

- (a) R
- (b) NR
- (c) R/N
- (d) R/N^2

(c). Even if other slots are idle.

Q4. The main timing overhead of circuit switching before data flows is:

- (a) Queuing delay
- (b) Circuit establishment (about one RTT)
- (c) Checksum computation
- (d) Retransmission

(b).

Q5. Which is a con of circuit switching?

- (a) Unpredictable performance
- (b) Needs congestion control
- (c) Inefficient for bursty traffic
- (d) Can route around failures

(c). (a), (b) are packet-switching cons; (d) is a packet-switching pro.

Q6. “Circuit switching doesn't route around trouble” means:

- (a) Circuits are slower
- (b) If a switch/link on the circuit fails, the circuit fails

- (c) Circuits can't cross borders
- (d) Circuits need IP addresses

(b).

Q7. In packet switching each packet:

- (a) Follows a pre-reserved path
- (b) Carries its destination and is handled independently
- (c) Needs a setup phase
- (d) Is never queued

(b).

Q8. Admission control in packet switching is performed:

- (a) Per connection
- (b) Per packet
- (c) Never
- (d) Only at the source

(b). A full buffer drops that packet; circuits admit per connection.

Q9. A virtual circuit:

- (a) Is pure circuit switching
- (b) Emulates circuit switching using packets with short circuit IDs
- (c) Is a wireless link
- (d) Uses no switches

(b). Examples: ATM, MPLS.

Q10. Store-and-forward means:

- (a) Routers store packets forever
- (b) A router receives the whole packet before forwarding
- (c) Bits are forwarded as they arrive
- (d) Routes are stored, not packets

(b). Each hop adds L/R .

Q11. Sending P packets over N store-and-forward links (equal rate, no queuing, ignore propagation) takes:

- (a) $NP L/R$
- (b) $(N + P - 1)L/R$
- (c) $(N + P)L/R$
- (d) PL/R

(b). Pipelining.

Q12. A protocol consists of:

- (a) Hardware and software
- (b) Message format and actions
- (c) Headers and trailers only
- (d) IP and TCP

(b).

Q13. In the slide’s Zoom example, saying “can you hear me?” after 3 s of silence corresponds to:

- (a) Encoding
- (b) A timeout action
- (c) Routing
- (d) Multiplexing

(b). Actions include what to do on *not* receiving.

Q14. Internet protocols are standardised by:

- (a) IEEE via RFCs
- (b) IETF via RFCs
- (c) ISO via RFCs
- (d) W3C via RFCs

(b). IEEE handles link layers like 802.11.

Q15. The network layer provides:

- (a) Best-effort local delivery
- (b) Best-effort global delivery
- (c) Reliable delivery
- (d) Useful user functions

(b). Local = link.

Q16. Which layer provides guarantees to applications?

- (a) Link
- (b) Network
- (c) Transport
- (d) Physical

(c).

Q17. Routers (in the basic model) implement:

- (a) All layers
- (b) Network and link layers
- (c) Only application
- (d) Only transport

(b).

Q18. The transport-layer PDU is called a:

- (a) Frame
- (b) Segment
- (c) Bit
- (d) Message

(b). Frame = link, datagram = network.

Q19. A benefit of layering is:

- (a) No header overhead
- (b) Replacing one layer's protocol without affecting others
- (c) Faster propagation
- (d) No need for standards

(b).

Q20. Transmission delay is measured from:

- (a) First bit at sender to last bit at receiver
- (b) First bit at sender to last bit at sender
- (c) Last bit at sender to last bit at receiver
- (d) Packet arrival to departure at router

(b). (a) is total packet delay; (c) is propagation.

Q21. Propagation delay depends on:

- (a) Packet size
- (b) Link bandwidth
- (c) Link length and signal speed
- (d) Queue length

(c).

Q22. The BDP of a 10 Gbps link with 10 ms propagation is:

- (a) 10^6 bits
- (b) 10^7 bits
- (c) 10^8 bits
- (d) 10^{10} bits

(c). = 12.5 MB.

Q23. A 100-byte packet on a 1 Mbps, 1 ms link is fully received after:

- (a) 1 ms
- (b) 0.8 ms
- (c) 1.8 ms
- (d) 8 ms

(c).

Q24. Upgrading from 1 Mbps to 1 Gbps for a single 100 B packet on a 1 ms link mainly shows that:

- (a) Delay drops 1000×
- (b) Propagation now dominates; delay barely changes
- (c) Queuing disappears
- (d) Throughput halves

(b). $1.8 \rightarrow 1.0008$ ms.

Q25. In the pipe view, a packet's area equals:

- (a) Its transmission time
- (b) Its size in bits
- (c) The BDP

(d) The propagation delay

(b). Height $R \times$ width L/R .

Q26. Queues in routers:

- (a) Eliminate loss
- (b) Absorb transient bursts but add delay
- (c) Reduce propagation delay
- (d) Exist only under persistent overload

(b).

Q27. Under persistent overload, a router eventually:

- (a) Increases link rate
- (b) Drops packets
- (c) Reorders packets
- (d) Sets up a circuit

(b).

Q28. Queuing delay is best characterised by:

- (a) A single fixed number
- (b) Statistical measures: mean, variance, tail probability
- (c) Link length
- (d) Packet size alone

(b).

Q29. Little's law is:

- (a) $L = A/W$
- (b) $L = A \times W$
- (c) $W = A \times L$
- (d) $A = L \times W$

(b).

Q30. Little's law requires:

- (a) Poisson arrivals
- (b) FIFO service
- (c) Exponential service times
- (d) Only a stationary system with long-term averages

(d).

Q31. Average 4 customers in the system, 2 arrivals every 3 min. Average wait:

- (a) 1.5 min
- (b) 2.67 min
- (c) 6 min
- (d) 8 min

(c). $4/(2/3)$.

Q32. Which delay is usually assumed negligible?

- (a) Transmission
- (b) Propagation
- (c) Queuing
- (d) Processing

(d).

Q33. Traffic intensity $\lambda a/R$ greater than 1 means:

- (a) Low delay
- (b) The queue grows without bound
- (c) The link is idle
- (d) Propagation dominates

(b).